# Chapter 11. Tree-Based Methods and Ensembles

Lecture 8 (regression and classification trees, pruning, bagging, OOB, random forests, boosting, AdaBoost, GBM, XGBoost, LightGBM, CatBoost, importance, SHAP)

## Contents

**1 Decision Trees** **2**

**2 Regression Trees** **2**

2.1 Example: Hitters data . . . 2

2.2 Building a tree . . . 2

2.3 Recursive binary splitting . . . 3

2.4 Pruning . . . 3

**3 Classification Trees** **4**

3.1 Trees versus linear models . . . 5

3.2 Advantages and disadvantages of trees . . . 5

**4 Bagging** **5**

4.1 Idea . . . 5

4.2 Out-of-bag error estimation . . . 6

**5 Random Forests** **6**

**6 Boosting** **6**

6.1 Boosting regression trees . . . 6

6.2 AdaBoost . . . 7

6.3 Gradient boosting machines . . . 7

6.4 XGBoost, LightGBM, CatBoost . . . 8

**7 Variable Importance** **8**

7.1 Impurity-based importance . . . 8

7.2 Permutation importance . . . 8

7.3 SHAP values . . . 9

# 1 Decision Trees

Decision trees apply to both regression and classification. They *stratify* or segment the predictor space into simple regions; since the splitting rules can be summarized in a tree, these are decision-tree methods.

**Pros and cons.** Trees are simple and useful for interpretation, but typically not competitive with the best supervised methods in accuracy. Bagging, random forests and boosting grow many trees and combine them into a consensus prediction, often dramatically improving accuracy — at the price of interpretability.

# 2 Regression Trees

## 2.1 Example: Hitters data

Predict the log salary of a baseball player from `Years` (in the major leagues) and `Hits` (in the previous year). The tree has two internal nodes and three terminal nodes (leaves):

$$R_1 = \{X : \texttt{Years} < 4.5\}, \quad R_2 = \{X : \texttt{Years} \geq 4.5,\ \texttt{Hits} < 117.5\},$$

$$R_3 = \{X : \texttt{Years} \geq 4.5,\ \texttt{Hits} \geq 117.5\},$$

and the number in each leaf is the mean response of the observations there. The regions are **terminal nodes**; the split points are **internal nodes**; trees are drawn upside down with leaves at the bottom.

**Interpretation.** Years is the most important factor: less experienced players earn less. Among inexperienced players, Hits plays little role; among players with five or more years, more Hits means higher salary. This is surely an oversimplification, but compared with a regression model it is easy to display, interpret and explain — and it contains an interaction that a linear model would need to be told about.

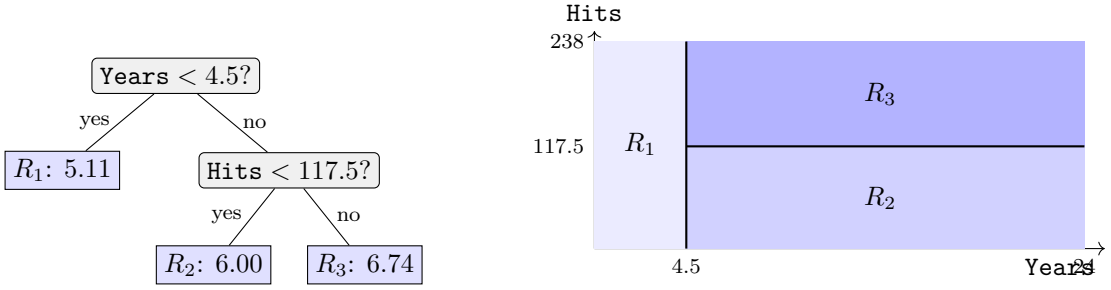



**Figure 1:** The Hitters regression tree (leaf values are mean log salaries) and the corresponding partition of the predictor space into three boxes.

## 2.2 Building a tree

1. Divide the predictor space into $J$ distinct, non-overlapping regions $R_1, \ldots, R_J$.
2. For every observation in $R_j$, predict the mean response of the training observations in $R_j$.

In theory the regions could have any shape; for simplicity and interpretability we use high-dimensional rectangles (boxes). The goal is to find boxes minimizing

$$\sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2,$$

where $\hat{y}_{R_j}$ is the mean response in box $j$ (for fixed boxes the mean is the RSS-optimal constant).

### 2.3 Recursive binary splitting

Considering every partition into $J$ boxes is computationally infeasible, so we use a **top-down, greedy** approach: top-down because it starts at the top of the tree and successively splits; greedy because at each step the best split is made *at that step*, without looking ahead.

**Algorithm: recursive binary splitting**

Select the predictor $X_j$ and cutpoint $s$ that minimize

$$\sum_{i:x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2,$$

$$R_1(j,s) = \{X : X_j < s\}, \; R_2(j,s) = \{X : X_j \geq s\}.$$

Then repeat within one of the resulting regions (not the whole space), and so on, until a stopping criterion is reached (e.g. no region contains more than five observations).

For each $j$ the search over $s$ is fast: sort the values and update running sums. A test observation is predicted by the mean of the training observations in its region.

### 2.4 Pruning

The process may predict the training set well but overfit. A smaller tree (fewer regions) may have lower variance and better interpretation at the cost of a little bias. Growing only while each split decreases RSS by more than a (high) threshold is too short-sighted: a seemingly worthless split early on may be followed by a very good split. Better: grow a very large tree $T_0$ and prune it back.

**Definition: cost-complexity (weakest-link) pruning**

For each $\alpha \geq 0$ there is a subtree $T \subset T_0$ minimizing

$$\sum_{m=1}^{|T|} \sum_{i:x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|,$$

where $|T|$ is the number of terminal nodes, $R_m$ the rectangle of the $m$th terminal node and $\hat{y}_{R_m}$ its mean.

$\alpha$ trades the subtree's complexity against its fit: $\alpha = 0$ gives $T_0$; as $\alpha$ increases there is a price for many leaves and the optimal subtree shrinks. As $\alpha$ increases, branches are pruned in a nested, predictable fashion, so the whole sequence of subtrees is easy to obtain: collapsing the internal node $t$ (with branch $T_t$) pays off once $\alpha \geq \{R(t) - R(T_t)\}/(|T_t| - 1)$, and the weakest link (smallest such value) is pruned first.

**Algorithm: building a regression tree**

1. Grow a large tree by recursive binary splitting, stopping only when each terminal node has fewer than some minimum number of observations.
2. Apply cost-complexity pruning to obtain the sequence of best subtrees as a function of $\alpha$.
3. Choose $\alpha$ by $K$-fold CV: for each $k$, repeat steps 1–2 on all but the $k$th fold and evaluate

the MSE on the $k$th fold as a function of $\alpha$; average and pick $\hat{\alpha}$.

4. Return the subtree from step 2 corresponding to $\hat{\alpha}$.

**Hitters**: 132 training and 131 test observations; a large tree was grown and pruned, and six-fold CV estimated the MSE as a function of $\alpha$. The CV error is minimized by a tree with three leaves, close to the test-error minimum.

## 3 Classification Trees

A classification tree predicts that each observation belongs to the most commonly occurring class of training observations in its region, and the class proportions serve as probability estimates. It is grown by recursive binary splitting, but RSS cannot be used as the criterion.

**Definition: node impurity measures**

With $\hat{p}_{mk}$ the proportion of training observations in region $m$ from class $k$:

$$E = 1 - \max_k \hat{p}_{mk} \text{ (classification error)}, \qquad G = \sum_{k=1}^{K} \hat{p}_{mk}(1-\hat{p}_{mk}) \text{ (Gini)},$$

$$D = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk} \text{ (cross-entropy)}.$$

The classification error is not sensitive enough for tree growing. The Gini index is a measure of total variance across the $K$ classes (the sum of Bernoulli variances $\hat{p}_{mk}(1-\hat{p}_{mk})$); it is small when all $\hat{p}_{mk}$ are close to 0 or 1, hence a measure of **node purity**. Cross-entropy behaves very similarly numerically.

**Example: why Gini is more sensitive than the error rate**

A node with 400 cases of each class is split either into $(300, 100)$ and $(100, 300)$, or into $(200, 400)$ and $(200, 0)$. Both splits have misclassification rate 0.25. The Gini values are $\frac{1}{2}(0.375) + \frac{1}{2}(0.375) = 0.375$ and $\frac{3}{4}(0.444) + \frac{1}{4}(0) = 0.333$: Gini prefers the second split, which creates a pure node.

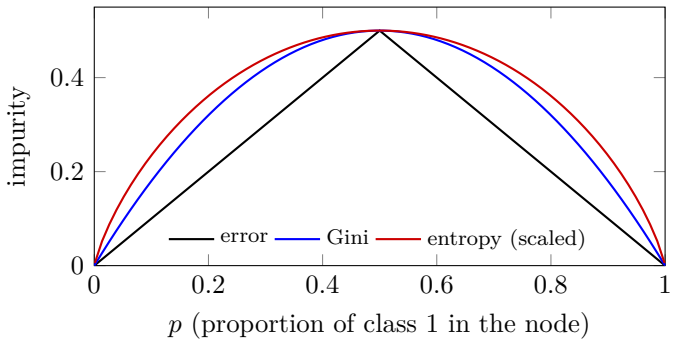



**Figure 2:** Node impurity for two classes. Gini $2p(1-p)$ and entropy (scaled to 0.5 at $p = \frac{1}{2}$) are strictly concave, so they reward splits that make nodes purer even when the error rate $\min(p, 1-p)$ does not change.

Because Gini and entropy are strictly concave in $p$, any split weakly decreases the weighted impurity (Jensen's inequality), whereas the piecewise-linear error rate often shows no improvement. For pruning, the classification error is preferred if prediction accuracy is the goal.

**Heart data**: binary outcome HD for 303 patients with chest pain (13 predictors: Age, Sex, Chol, ...). CV yields a tree with six terminal nodes. Trees handle qualitative predictors directly (a split assigns some levels to one branch).

### 3.1 Trees versus linear models

Linear regression assumes $f(X) = \beta_0 + \sum_j \beta_j X_j$; a regression tree assumes $f(X) = \sum_{m=1}^{M} c_m \mathbb{I}\{X \in R_m\}$. If the true relationship is close to linear, the linear model wins; if it is highly nonlinear and box-like, the tree wins. Example: the green class has two independent standard normal inputs, the red class is conditioned on $X_1^2 + X_2^2 \geq 4.6$; the circular boundary cannot be captured linearly, while a tree approximates it by a staircase of boxes.

### 3.2 Advantages and disadvantages of trees

Trees are very easy to explain (even easier than linear regression), may mirror human decision-making, can be displayed graphically and interpreted by non-experts, and handle qualitative predictors without dummy variables. But they generally lack the predictive accuracy of other methods and are non-robust: a small change in the data can cause a large change in the tree. Aggregating many trees substantially improves performance.

## 4 Bagging

### 4.1 Idea

Bootstrap aggregation (bagging) is a general-purpose procedure for reducing the variance of a learning method, particularly useful for trees. Given $n$ independent observations each with variance $\sigma^2$, the mean has variance $\sigma^2/n$: averaging reduces variance. We do not have many training sets, so we bootstrap: generate $B$ bootstrapped training sets, fit $\hat{f}^{*b}(x)$ on the $b$th, and average,

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}^{*b}(x).$$

For classification, record the class predicted by each of the $B$ trees and take a **majority vote** (or average the class probabilities). Bagging can improve tree predictions a lot (and gives more flexible decision boundaries), but interpretation is harder. The trees are grown deep and not pruned: each has high variance and low bias, and averaging reduces the variance.

**Theorem: variance of an average of identically distributed predictors**

If $\hat{f}^{*1}, \ldots, \hat{f}^{*B}$ at a point $x$ each have variance $\sigma^2$ and pairwise correlation $\rho$, then

$$\operatorname{Var}\Big(\frac{1}{B} \sum_{b} \hat{f}^{*b}(x)\Big) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

**Proof.** $\operatorname{Var}(\sum_b \hat{f}^{*b}) = B\sigma^2 + B(B-1)\rho\sigma^2$; divide by $B^2$. ■

As $B$ grows the second term vanishes but the first remains: bagged trees built on overlapping bootstrap samples are positively correlated, which limits the gain. Increasing $B$ does not lead to overfitting.

### 4.2 Out-of-bag error estimation

**Theorem: each bagged tree uses about two thirds of the observations**

The probability that a given observation is *not* in a bootstrap sample is $(1-1/n)^n \to e^{-1} \approx 0.368$.

**Proof.** Each of the $n$ independent draws misses it with probability $1-1/n$; and $n\log(1-1/n) \to -1$. ■

The remaining third, the **out-of-bag** (OOB) observations of a tree, can be predicted by it. Each observation $i$ is OOB for about $B/3$ trees; averaging (or voting over) those predictions gives an OOB prediction for $i$, and the resulting OOB error is a valid estimate of the test error. For large $B$ it is essentially the LOOCV error of bagging, obtained without refitting. On the Heart data, the OOB errors of bagging and random forests track their test errors as functions of $B$, and both beat a single tree.

## 5 Random Forests

Random forests improve bagged trees by a small tweak that **decorrelates** the trees. As in bagging, trees are built on bootstrapped samples; but each time a split is considered, a random sample of $m$ predictors is chosen as split candidates from the full set of $p$, and the split may use only one of them. A fresh sample is taken at each split; typically $m \approx \sqrt{p}$ (classification) or $p/3$ (regression).

Why it helps: with one very strong predictor, most bagged trees use it in the top split, so they look alike and their predictions are highly correlated; averaging highly correlated quantities reduces variance little ($\rho\sigma^2$ term). Forcing each split to consider a subset gives other predictors a chance, lowering $\rho$. $m = p$ is bagging.

**Gene-expression data**: 4,718 genes measured on 349 patients, 15 classes (normal or one of 14 cancer types). Using the 500 genes with largest training variance, random forests with $m < p$ improve slightly over bagging ($m = p$); a single tree has error 45.7%. As with bagging, random forests do not overfit as $B$ increases, so a sufficiently large $B$ is recommended.

## 6 Boosting

### 6.1 Boosting regression trees

Like bagging, boosting is a general approach. Bagging fits a separate tree to each bootstrap copy, independently of the other trees. Boosting grows the trees **sequentially**: each tree uses information from previously grown trees, and no bootstrap is used.

**Algorithm: boosting for regression trees**

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all $i$.
2. For $b = 1, \ldots, B$: fit a tree $\hat{f}^b$ with $d$ splits ($d+1$ leaves) to the data $(X, r)$; update $\hat{f}(x) \leftarrow \hat{f}(x) + \lambda\hat{f}^b(x)$ and the residuals $r_i \leftarrow r_i - \lambda\hat{f}^b(x_i)$.
3. Output $\hat{f}(x) = \sum_{b=1}^{B} \lambda\hat{f}^b(x)$.

Instead of fitting one large tree (fitting the data hard, potentially overfitting), boosting **learns slowly**: small trees fitted to the residuals improve $\hat{f}$ gradually in areas where it does poorly,

and the shrinkage $\lambda$ slows the process further, letting more and different-shaped trees attack the residuals.

**Three tuning parameters.**

- The number of trees $B$: unlike bagging and random forests, boosting can overfit if $B$ is too large (slowly); choose $B$ by CV.
- The shrinkage $\lambda$: the learning rate, typically 0.01 or 0.001; very small $\lambda$ needs very large $B$.
- The number of splits $d$: the interaction depth. Often $d = 1$ works well (stumps, giving an additive model); $d$ splits allow interactions of up to $d$ variables.

On the gene-expression data, boosted stumps ($\lambda = 0.01$) slightly outperform depth-2 boosting and random forests.

## 6.2 AdaBoost

**Algorithm: AdaBoost, $y \in \{-1, +1\}$**

Start with weights $D_1(i) = 1/n$. For $b = 1, \ldots, B$:
(i) train a weak learner $h_b$ using weights $D_b$;
(ii) compute its weighted error $\epsilon_b = \sum_i D_b(i)\mathbb{I}\{h_b(x_i) \neq y_i\}$;
(iii) set $\alpha_b = \frac{1}{2}\log\frac{1-\epsilon_b}{\epsilon_b}$;
(iv) update $D_{b+1}(i) = D_b(i)\exp\{-\alpha_b y_i h_b(x_i)\}/Z_b$, $Z_b$ a normalizer. Output $H(x) = \text{sign}(\sum_{b=1}^{B}\alpha_b h_b(x))$.

Misclassified observations get larger weights in the next round, so later learners concentrate on the hard cases; accurate learners get a larger vote.

**Theorem: AdaBoost minimizes the exponential loss stagewise**

With $F_{b-1} = \sum_{s<b}\alpha_s h_s$, the pair $(\alpha_b, h_b)$ minimizes $\sum_i \exp\{-y_i(F_{b-1}(x_i) + \alpha h(x_i))\}$.

**Proof.** With $D_b(i) \propto e^{-y_i F_{b-1}(x_i)}$, the objective is proportional to $e^{-\alpha}(1-\epsilon) + e^{\alpha}\epsilon$, which for $\alpha > 0$ is increasing in the weighted error $\epsilon$ of $h$ (so $h_b$ minimizes it), and setting the $\alpha$-derivative to zero gives $e^{2\alpha} = (1-\epsilon)/\epsilon$. The new weights are $e^{-y_i F_b(x_i)}$, i.e. the update rule. ■

In the classic toy example the three stumps have $\epsilon = 0.30, 0.21, 0.14$ and votes $0.42, 0.65, 0.92$; their weighted combination gives a boundary no single stump can draw.

## 6.3 Gradient boosting machines

GBM views boosting as gradient descent in function space with an arbitrary differentiable loss $L$:

(i) at step $b$ compute pseudo-residuals $r_{ib} = -\partial L(y_i, F(x_i))/\partial F(x_i)$ at $F = F_{b-1}$;

(ii) fit a tree to them;

(iii) update $F_b = F_{b-1} + \lambda h_b$. For squared loss $\frac{1}{2}(y-F)^2$ the pseudo-residual is the ordinary residual $y - F$, recovering the algorithm above (the slide writes $y = f_b(x) + R_b$, $R_b = y - f_b(x)$, and the next tree targets $R_b$).

| Problem | Loss | Negative gradient |
|---|---|---|
| regression | squared $\frac{1}{2}(y-F)^2$; absolute $\lvert y-F\rvert$; Huber | $y - F$; $\text{sign}(y-F)$; clipped residual |
| classification | Bernoulli deviance $\log(1+e^{-2yF})$; exponential $e^{-yF}$ | $2y/(1+e^{2yF})$; $ye^{-yF}$ |

Standard GBM is powerful but can overfit with too many trees, has no explicit regularization of tree complexity, and sequential fitting can be computationally inefficient.

### 6.4 XGBoost, LightGBM, CatBoost

**XGBoost** is an optimized, regularized gradient boosting with objective

$$\mathcal{L} = \sum_{i=1}^{n} L(y_i, \hat{y}_i) + \sum_{b=1}^{B} \Omega(f_b), \qquad \Omega(f_b) = \gamma T_b + \frac{\lambda}{2} \sum_{j=1}^{T_b} w_{bj}^2,$$

where $T_b$ is the number of leaves of tree $f_b$ and $w_{bj}$ the score of leaf $j$. A second-order Taylor expansion with gradients $g_i$ and Hessians $h_i$ gives the optimal leaf weight $w_j^* = -\sum_{i \in I_j} g_i / (\sum_{i \in I_j} h_i + \lambda)$ and a closed-form split gain. It improves regularization (penalizing complex trees), optimization (gradients and Hessians) and systems (parallelisation, efficient pruning, cache-aware computation).

**LightGBM** targets scalability: histogram-based learning bins continuous features to speed up split finding; leaf-wise growth expands the leaf with the largest loss reduction (reducing loss more aggressively); it cuts memory and training time on large, high-dimensional data.

**CatBoost** targets categorical data: native categorical handling via target statistics computed in a principled (ordered) way, and *ordered boosting*, which prevents target leakage and prediction shift; it is robust and less prone to overfitting on small data.

| Method | Main focus | Mechanism |
|---|---|---|
| XGBoost | regularization and second-order optimization | controls tree complexity; uses gradients and Hessians |
| LightGBM | speed and scalability | histogram-based splitting, leaf-wise growth |
| CatBoost | categorical data, bias reduction | ordered target statistics and ordered boosting |

All three are improved versions of GBM.

## 7 Variable Importance

### 7.1 Impurity-based importance

Each split reduces a loss (RSS, Gini); a variable is important if it appears often in splits and produces large improvements. For tree-based models

$$\text{Imp}(X_j) = \sum_{\text{splits on } X_j} \text{reduction in loss}, \qquad \text{final importance} = \frac{1}{B} \sum_{b=1}^{B} \text{Imp}_b(X_j),$$

using RSS for regression and Gini or cross-entropy for classification. On the Heart data, the largest mean decreases in Gini belong to Thal, Ca and ChestPain. Importance gives a global view of the model (variable selection, interpretation, feature engineering) but does not capture causal relationships.

**Limitations**: biased toward variables with many possible splits (continuous or high-cardinality); depends on the model structure and configuration; not comparable across models. We need model-agnostic, more reliable measures.

### 7.2 Permutation importance

Break the relationship between $X_j$ and $Y$ by randomly shuffling $X_j$ and measure the increase in prediction error:

$$\text{Imp}(X_j) = \text{Error}_{\text{perm}(j)} - \text{Error}_{\text{original}}.$$

It is model-agnostic and tied directly to predictive performance (compute it on OOB or validation data; with correlated predictors, a substitute variable can mask the importance of the shuffled one).

### 7.3 SHAP values

SHAP (Shapley additive explanations) distributes a prediction fairly among the features:

$$f(x) = \phi_0 + \sum_{j=1}^{p} \phi_j, \qquad \phi_j = \sum_{S \subseteq \{1,\dots,p\} \setminus \{j\}} \frac{|S|!\,(p - |S| - 1)!}{p!} \big[v(S \cup \{j\}) - v(S)\big],$$

where $v(S) = \mathbb{E}[f(X) \mid X_S = x_S]$ is the prediction using only the features in $S$ and $\phi_0 = \mathbb{E}f(X)$. $\phi_j$ is the average marginal contribution of $X_j$ over all orders in which features can be added. Shapley values are the unique attribution satisfying efficiency ($\sum_j \phi_j = f(x) - \phi_0$), symmetry, dummy and additivity, which makes them consistent and theoretically justified; they give *local* explanations per observation (TreeSHAP computes them efficiently for trees).

| Method | Type | Characteristic |
|---|---|---|
| tree importance | model-specific | fast but biased |
| permutation | model-agnostic | based on prediction error |
| SHAP | model-agnostic | local and consistent explanations |

## Summary

- Trees split the predictor space by recursive binary splitting (RSS for regression, Gini/entropy for classification) and are pruned by cost complexity with CV.
- Bagging averages deep bootstrap trees; OOB observations (about one third) estimate test error; random forests decorrelate trees by sampling $m$ predictors at each split.
- Boosting fits small trees sequentially to residuals with shrinkage; $B$, $\lambda$, $d$ are tuning parameters; AdaBoost minimizes exponential loss, GBM follows the negative gradient; XGBoost, LightGBM, CatBoost refine GBM.
- Importance: impurity-based (fast, biased), permutation (model-agnostic), SHAP (local, axiomatic).